

Joint payload and embedded index codes enable indel-resilient DNA data storage

Table of Contents

- About the Project
- Project Structure
- NGS_Decoder_ECS_Pool2
  - Overview
  - Pipeline
  - Code Layout
  - Usage Example for NGS Decoder
- ONT_Rapid_Decoder of ECS-Pool2
  - Overview
  - Pipeline
  - Code Layout
  - Usage Example for ECS-Pool2
- ONT_Rapid_Decoder of IPS-Pool
  - Overview
  - SRC
  - ONT_FASTQ
  - Pipeline
  - Code Layout
  - Usage Example for IPS-Pool
- Dependencies
- License

About the Project

Synthetic oligonucleotide pools are a promising medium for long-term archival data storage, but reliable readout remains challenging under severe synthesis and sequencing errors. This project implements a joint payload and embedded index code (JPEIC) framework that conceals indices via position hiding and value masking. Single-strand error correction is incorporated to ensure robustness against insertion, deletion, and substitution errors during synthesis and sequencing.

Experimental verification was conducted across two synthesis technologies—electrochemical synthesis and inkjet printing synthesis—and two sequencing platforms, including next-generation sequencing (NGS) and nanopore sequencing. Accordingly, this project provides code for decoding and data readout from both NGS and nanopore sequencing data.

1. **Large-scale ECS pool + NGS readout**: NGS-based readout from large-scale electrochemically synthesized oligo pools.
2. **Nanopore rapid readout**: Online decoding pipelines for nanopore sequencing. Two nanopore sequencing schemes are included:

- **ECS-Pool2 (electrochemically synthesized 1M oligo pool): rapid decoding on a PromethION 2 Solo sequencer**
- **IPS-Pool3 (inkjet printing synthesized oligo pool): rapid decoding on MinION sequencer**

Project Structure

```
/
├── NGS_Decoder_ECS_pool/              # Large-scale DNA pool offline decoding
system
│   ├── configuration_files/           # Primers, watermark, interleave order
│   ├── include/                       # Headers
│   ├── src/                           # C/C++ sources for NGS readout
│   ├── Assembled_fastq/               # Input NGS data directory
│   │   └── assembled_test.fastq       # Input assembled FASTQ file
│   └── run_ngs_recovery.sh            # Main offline readout script
│
├── ONT_Rapid_Decoder_ECS_Pool2/       # Rapid nanopore readout (ECS-Pool2, 1M
pool)
│   ├── configuration_files/           # Primers, watermark, interleave order
│   ├── include/                       # Headers
│   ├── src/                           # C/C++ sourcess
│   ├── minimap2                       # Local minimap2 exe
│   ├── ECS-Pool2_nanopore_data        # Nanopore sequencing data
│   └── run_video_rapid_recovery.sh    # Two-chip online readout pipeline
│
├── ONT_Rapid_Decoder_IPS_Pool/        # Rapid nanopore decoding system
│   ├── SRC/                           # Decoding programs for nanopore data
│   │   ├── run_decoder.sh             # Main decoder script
│   │   └── [decoder executables]      # Rapid decoding programs
│   └── ONT_FASTQ/                     # Nanopore sequencing data
│       ├── file_001.fastq             # Sample nanopore reads (100 reads each)
│       ├── file_002.fastq
│       ├── ...
│       └── file_400.fastq             # 400 files total as examples
│
└── README.md                          # This documentation
```

NGS_Decoder_ECS_Pool2

Overview

The **NGS_Decoder_ECS_Pool2** module implements an offline readout pipeline designed to recover data from paired-end assembled FASTQ files generated by Next-Generation Sequencing (NGS). It leverages the high throughput and lower error rate of NGS to reliably reconstruct large-scale data pools.

The pipeline performs primer trimming, robust error correction (Forward-Backward algorithm + BCH decoding), data de-interleaving, and final Reed-Solomon decoding to retrieve the original digital file (MP4 video).

Pipeline

The offline pipeline in `run_video_readout_NGS.sh` follows a sequential 5-step workflow:

1. **Primer Alignment**:
    - **Input**: `assembled_test.fastq`, primer list.
    - **Action**: Identify reads containing valid 5' and 3' primers, trim them to the payload region, and perform length checks.
    - **Output**: `primer_alignment_result.fasta` (payload sequences).
2. **FB + BCH Decoding**:
    - **Input**: Payload sequences.
    - **Action**: Apply the joint payload and embedded index code (JPEIC) decoding algorithm.
        - **Forward-Backward (FB) Algorithm**: Correct insertion/deletion errors based on Forward-backward algorithm.
        - **BCH Decoding**: Correct substitution errors and verify data integrity via CRC/parity checks.
    - **Output**: `BCH_decoded_bits.txt` (Recovered bitstreams with addresses).
3. **Majority voting and De-interleaving**:
    - **Input**: Decoded BCH bitstreams.
    - **Action**: perform majority voting and Map the linear bitstreams back into the 2D matrix structure using a symbol interleaving pattern to disperse burst errors.
    - **Output**: `codeword_for_rs_decoding/` (Deinterleaved symbols organized by RS coding units).
4. **RS Decoding**:
    - **Input**: Deinterleaved symbol groups.
    - **Action**: Check erasure rates for each group. If the erasure count is within the correction capability (<15%), parallel Reed-Solomon decoding is executed to recover the original information bits.
    - **Output**: `bits_for_video_recovery.txt` (Fully corrected binary stream).
5. **Video Recovery**:
    - **Input**: Corrected binary stream.
    - **Action**: reconstruct the original file (e.g., MP4).
    - **Output**: `recovered_video.mp4`.

Code Layout

```
NGS_Decoder_ECS_Pool2/
├── configuration_files/
│   ├── primer_list_10nt.txt
│   ├── watermark_448800bit.txt
│   ├── rs4000_250group_diagonal_interleave_order
│   └── SequenceLimitRunLength
├── Assembled_fastq/
│   └── assembled_test.fastq              # Hardcoded input file
```

```
├── include/
│   ├── fb_algorithm.h
│   ├── edlib.h
│   └── bch_decode_bm_fast.h
├── src/
│   ├── ngs_primer_align.c                # Step 1: Primer trimming
│   ├── indel_correction_BCH_decoding.c   # Step 2: FB + BCH decoding
│   ├── fb_algorithm.cpp
│   ├── bch_decode_bm_fast.c
│   ├── majority_voting_deinterleave.c    # Step 3: Majority_voting and De-
interleaving
│   ├── rs_decode_parallel.c              # Step 4: RS decoding
│   └── recover_video_from_bitstream.c    # Step 5: Video reconstruction
└── run_video_readout_NGS.sh              # Main execution script
```

Usage Example for NGS Decoder

1. **Navigate to the decoder directory:**

   ```
   cd NGS_Decoder_ECS_pool2
   ```

2. **Build the executables:**

   ```
   make
   ```

3. **Prepare Input Data:** Ensure your paired-end assembled FASTQ file is placed in the `Assembled_fastq` folder with the name `assembled_test.fastq`.The test dataset is available in the SRA database and can be obtained from `SRX32566051` under accession number `PRJNA1439609`.

4. **Run the offline recovery pipeline:**

   ```
   ./run_video_readout_NGS.sh
   ```

ONT_Rapid_Decoder of ECS-Pool2

Overview

The ONT_Rapid_Decoder of ECS-Pool2 module implements an online readout pipeline for a large electrochemically synthesized oligo pool (ECS-Pool2, 1,000,000 strands) on an ONT PromethION 2 Solo sequencer. The pipeline is designed for rapid nanopore streaming readout and supports a two-chip configuration, processing FASTQ batches as they are generated.

In the demonstrated setting, sequencing reads are continuously output in batches of 50,000 reads per FASTQ. As sequencing progresses, reads that pass JPEIC decoding and CRC verification are progressively reconstructed into a growing set of regenerative reference sequences. The expanding reference set enables increasingly efficient screening of incoming reads by alignment, so that JPEIC decoding is applied primarily to previously unseen strands, reducing computational burden at later stages.

A predefined decoding policy is applied: since the cross-strand RS code tolerates up to 15% strand dropout, cross-strand decoding is triggered once 85% of reference strands have been reconstructed.

Reported performance (manuscript description):

Reported performance (manuscript description):

- Platform: ONT PromethION 2 Solo (HAC mode)
- Pool: ECS-Pool2 (1,000,000 strands)
- File: 14.0 MB video
- Full recovery time: 18 min 44 s after sequencing initiation
- Sequencing speed: 5,371 reads/s
- Readout speed: 0.75 MB/min
- Additional preparation: ~1.5 h library preparation; 6.3 min flow-cell preparation

Pipeline

The online pipeline in `run_video_rapid_recovery.sh` follows a read-by-read accelerated workflow matched to nanopore streaming. Each loop processes one complete FASTQ batch (50,000 reads) from either chip:

1. **Primer alignment (ONT reads → valid payload reads)**
    - Input: FASTQ batch, primer list
    - Output: primer-trimmed reads in FASTA (~130 nt), match statistics, length records
2. **Reference-based filtering (exclude already-seen strands)**
    - Align current reads to the **cumulative regenerative references** using `minimap2`
    - Extract unmapped reads as “new candidate reads” for JPEIC decoding
3. **FB + BCH decoding (indel correction + CRC verification)**
    - Apply forward–backward decoding and BCH decoding for candidate reads
    - Output: decoded BCH bitstreams and newly recovered references (FASTA)
4. **Progressive merge of BCH decoding results**
    - Merge new BCH results into cumulative BCH records
    - Update cumulative regenerative references
    - Produce an RS-trigger flag according to decoding progress
5. **Deinterleave & RS decoding (triggered by policy)**
    - Deinterleave cross-strand RS symbols using a predefined interleaving order
    - Run parallel RS decoding when erasure/dropout meets the threshold (<15%)

6. **File reconstruction (bitstream → MP4)**
    - Descramble using the watermark sequence
    - Pack bits into bytes and write the recovered MP4 file

Code Layout

```
ONT_Rapid_Decoder_ECS_Pool2/
├── configuration_files/
│   ├── primer_list_10nt.txt
│   ├── watermark_448800bit.txt
│   ├── rs4000_250group_diagonal_interleave_order
│   └── [other config files]
├── ECS-Pool2_nanopore_data/
│   ├── sub_0.fastq
│   ├── sub_1.fastq
│   └── ...
├── include/
│   ├── edlib.h
│   ├── fb_algorithm.h
│   └── bch_decode_bm_fast.h
├── src/
│   ├── ont_primer_align.c
│   ├── edlib.cpp
│   ├── extract_unmapped_reads.c
│   ├── indel_correction_BCH_decoding.c
│   ├── fb_algorithm.cpp
│   ├── bch_decode_bm_fast.c
│   ├── merge_bch_results.c
│   ├── deinterleave_rs_symbols.c
│   ├── rs_decode_parallel.c
│   └── recover_video_from_bitstream.c
├── minimap2
└── run_video_rapid_recovery.sh
```

Usage Example for ECS-Pool2

1. **ECS-Pool2 rapid decoder directory**

```
cd "ONT_Rapid_Decoder of ECS-Pool2"
```

2. **Build all executables**

```
make
```

3. **Prepare the input directories** Download the complete nanopore sequencing dataset from the SRA database (`SRX32566052` under accession number `PRJNA1439609`). The uploaded dataset is provided as

compressed files. Before running the pipeline, execute `step1_run_split_sequence.sh` to split the sequencing data into multiple FASTQ files, for example with 200,000 reads per file.

- Expected layout:
  - `./ECS-Pool2_nanopore_data/sub_0.fastq`
  - `./ECS-Pool2_nanopore_data/sub_1.fastq`
  - `./ECS-Pool2_nanopore_data/sub_2.fastq`
  - `...`

4. **Run rapid recovery**

```
./run_video_rapid_recovery.sh
```

ONT_Rapid_Decoder of IPS-Pool

Overview

The ONT_Rapid_Decoder_of_IPS-Pool module provides rapid decoding pipeline for nanopore sequencing data from inkjet printing synthesized pools. This system demonstrates that our DNA storage framework can support online rapid data recovery, making it suitable for applications requiring immediate access to stored information.

SRC

The SRC directory contains the core decoding programs specifically designed for processing Oxford Nanopore Technologies (ONT) sequencing data.

ONT_FASTQ

The ONT_FASTQ directory contains example nanopore sequencing data used for testing and demonstration of the rapid decoding system.

- Format: FASTQ
- Read count per file: 100 reads
- Total files: 400
- Coverage: Designed to provide sufficient redundancy for reliable decoding
- Error patterns: Representative of real nanopore sequencing characteristics

Pipeline

The rapid pipeline in `rapid_process_4000pool.sh` follows a read-by-read accelerated workflow matched to nanopore sequencing reads. Each loop processes one complete FASTQ batch (100 reads):

1. **Primer alignment (ONT reads → valid payload reads)**
   - Input: FASTQ batch, primer list
   - Output: primer-trimmed reads in FASTA (~208 nt), match statistics, length records

2. **FB + LDPC Decoding**:
    - **Input**: Payload sequences.
    - **Action**: Apply the joint payload and embedded index code (JPEIC) decoding algorithm.
        - **Forward-Backward (FB) Algorithm**: Correct insertion/deletion errors based on Forward-backward algorithm.
        - **LDPC Decoding**: Correct substitution errors and verify data integrity via ldpc decoding.
    - **Output**: `ldpc_decoded_bits.txt` (Recovered bitstreams with addresses).
3. **Majority voting and De-interleaving**:
    - **Input**: Decoded LDPC bitstreams.
    - **Action**: perform majority voting and Map the linear bitstreams back into the 2D matrix structure.
    - **Output**: `result_for_rs_decoding.txt`.
4. **RS Decoding**:
    - **Input**: Deinterleaved symbol groups.
    - **Action**: Check erasure rates for each group. If the erasure count is within the correction capability (<2.5%), parallel Reed-Solomon decoding is executed to recover the original information bits.
    - **Output**: `bits_for_video_recovery.txt` (Fully corrected binary stream).
5. **Image Recovery**:
    - **Input**: Corrected binary stream.
    - **Action**: reconstruct the original file (.jpg).
    - **Output**: `recovered_pic3.jpg`.

Code Layout

```
ONT_Rapid_Decoder_IPS_Pool/
├── SRC/
│   ├── rapid_process_4000pool.sh      # Main execution script
│   ├── decoder_executable             # Compiled decoding logic
│   └── [other source files]
├── Processing_result/                 # The result folder
└── ONT_FASTQ/                         # Nanopore sequencing data input
├── file_001.fastq                     # Sample data chunk (100 reads)
├── file_002.fastq
└── ...
```

Usage Example for IPS-Pool

1. **Set up the nanopore data:**

```
# The ONT_FASTQ directory already contains 400 example files
```

```
# For rapid processing, you can replace these with live nanopore output
```

2. **Run the rapid decoder:**

```
cd ONT_Rapid_Decoder_IPS-Pool/SRC
./rapid_process_4000pool.sh
```

3. **Monitor rapid decoding:**

```
# The decoder will process files sequentially and output results
# Progress and quality metrics will be displayed during processing
# Results are saved in ONT_Rapid_Decoder_IPS-Pool/Processing_result directory
```

Dependencies

System Requirements

- **Operating System**: Linux (Tested on Ubuntu 18.04+)
- **Shell**: Bash shell environment
- **Compilers**:
  - `gcc` (C compiler)
  - `g++` (C++ compiler supporting C++11 standard)
- **Hardware**: Sufficient memory (RAM) for processing large sequencing FASTQ files.

External Tools & Libraries

This implementation leverages several third-party software packages under their respective licenses. Please ensure the following are installed or available:

- **minimap2**: Read-to-reference alignment for filtering already-reconstructed strands.
  - URL: https://github.com/lh3/minimap2
- **Edlib**: A lightweight and fast C/C++ library for sequence alignment using edit distance.
  - URL: https://github.com/Martinsos/edlib
- **LDPC Codes**: Low Density Parity Check encoder/decoder implementation by Radford M. Neal.
  - URL: https://github.com/radfordneal/LDPC-codes
- **RS Codes**: Reed–Solomon error correction codes by Robert Morelos-Zaragoza.
  - URL: https://www.eccpage.com
- **BCH Codes**: Bose–Chaudhuri–Hocquenghem codes (featuring the Berlekamp–Massey algorithm) by Robert Morelos-Zaragoza.
  - URL: http://the-art-of-ecc.com

Installation:

```
# Install required tools
git clone https://github.com/lh3/minimap2.git
cd minimap2 && make

git clone https://github.com/Martinsos/edlib.git
cd edlib && make
```

License

This project is licensed under the MIT License. See the LICENSE file for details.